IntelliGrid: Electricity Theft and Anomaly Detection

Oct 16, 2026 · @Shivam Pathak

A deep learning project that flags likely electricity theft from smart-meter data, using an LSTM classifier and an autoencoder anomaly detector, benchmarked against classical models on the real SGCC dataset.

Project summary

Problem. Electricity theft (meter tampering, bypassing, illegal connections) is a major source of non-technical loss for power utilities. A utility cannot physically inspect every home, so it needs a way to rank customers by risk and send inspectors where they are most likely to find theft. It is both a financial fraud problem (lost revenue) and a cybersecurity problem (tampered readings from networked smart meters).

Solution. IntelliGrid scores every customer two independent ways from their daily consumption history. A **supervised LSTM** reads the consumption sequence and learns from the dataset's theft labels to output a theft probability. A **dense autoencoder**, trained only on customers labelled normal, learns what ordinary usage looks like; customers it reconstructs poorly get a high **anomaly score**. Both are compared against classical models (Logistic Regression, Random Forest, Isolation Forest), and the results feed a customer investigation view.

Why it works. The two branches answer different questions: the LSTM catches theft patterns seen in the labels, while the autoencoder flags unusual behaviour even without labels. Every result is benchmarked against simpler models, so any gain from deep learning is measured, not assumed. The stack is lightweight and reproducible: Python, Pandas, NumPy, Scikit-learn, TensorFlow/Keras and Jupyter notebooks, runnable on Google Colab.

Progress so far

We have finalised the project scope, obtained the SGCC dataset, verified it is real utility data, and profiled it; the first EDA notebook cells are written and tested.

Property	Value (verified from the file)
Source	State Grid Corporation of China, released with Zheng et al., IEEE TII 2018

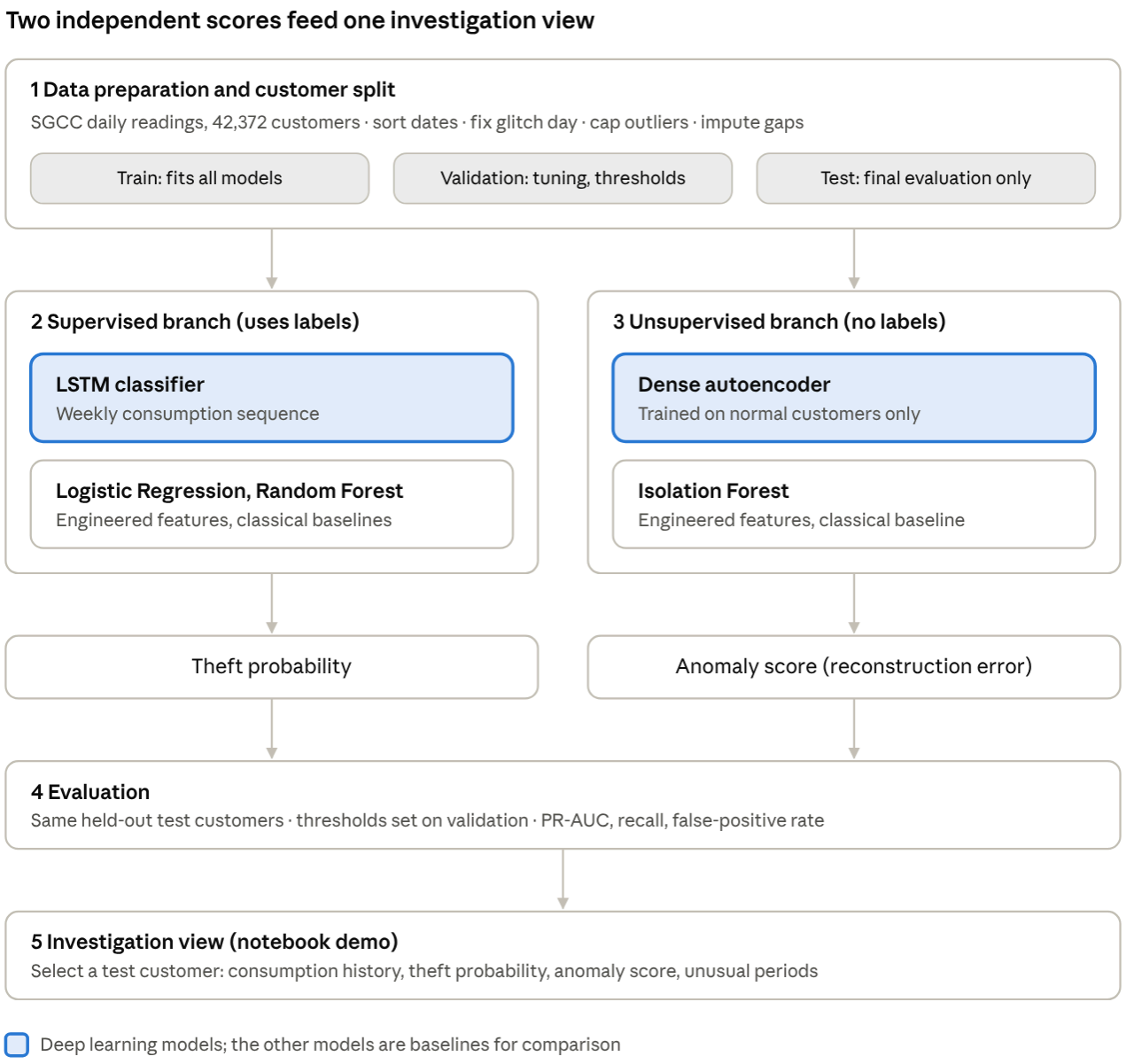
Property	Value (verified from the file)
Customers	42,372 (one row per customer, no duplicates)
Labels	3,615 theft (8.5%), 38,757 normal
Period	1 Jan 2014 to 31 Oct 2016, 1,034 daily columns (18 Sep 2016 absent)
Missing readings	25.6% overall; 31.5% for theft customers vs 25.1% for normal
Typical usage	median 4.6 kWh per customer per day

Findings that shape our design:

- **Real, not synthetic.** The data carries real metering faults: a system-wide glitch on 3 Oct 2014 (median reading jumps from 4.5 to 144 kWh), readings up to 800,003 kWh in a day, and missingness that falls from about 36% in 2014–15 to under 2% in 2016 as meters came online.
- **Ordering traps.** Date columns are stored in text order (2014/1/1, 2014/1/10, 2014/1/11…), and all theft customers are listed first. Both must be fixed before any sequence modelling or splitting.
- **Weak weekly cycle.** Median usage is almost flat across weekdays, and theft customers are not less regular than normal ones. We therefore do not claim that theft "breaks a weekly rhythm"; models must learn the difference from data.
- **A baseline floor.** Average consumption alone separates theft from normal customers with ROC-AUC 0.66. Any model must clearly beat this to be worth its complexity.

Next step: complete Notebook 01 (missing values, class imbalance, consumption trends) and begin preprocessing.

High-level diagram

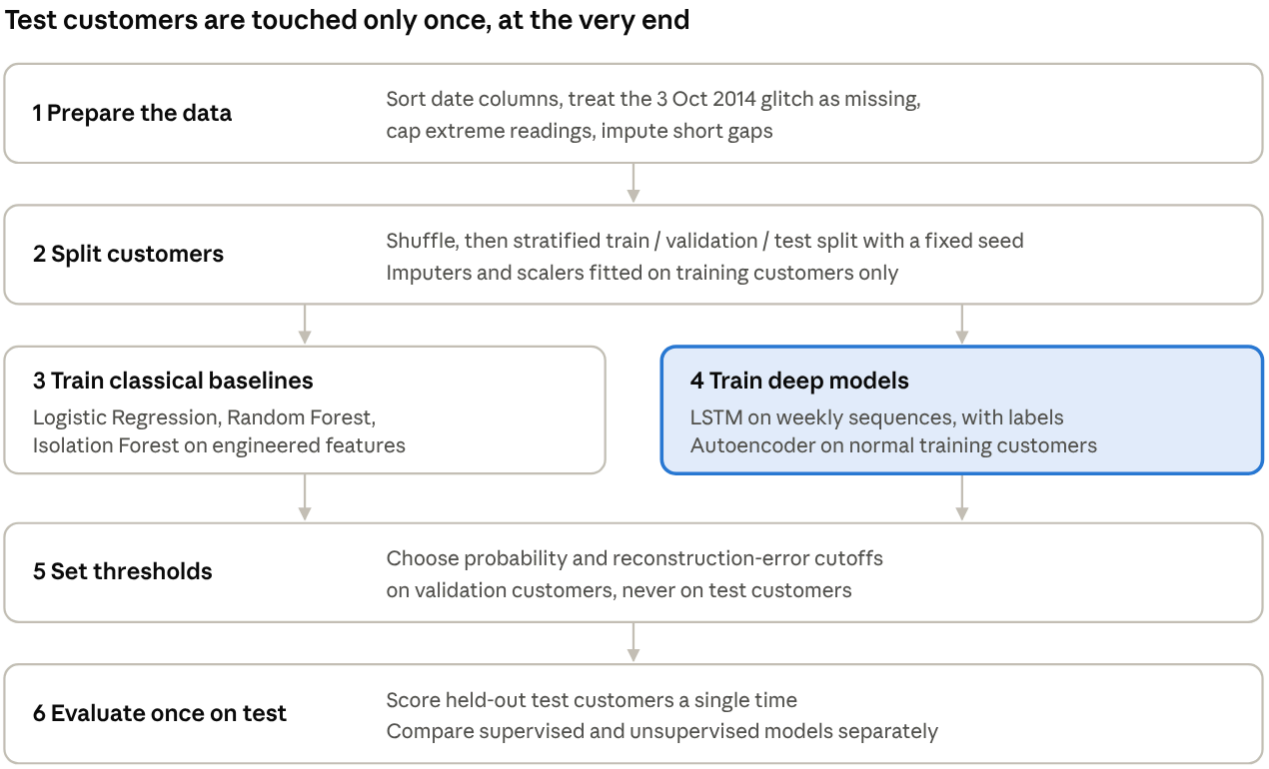


IntelliGrid architecture · data split, 2 branches, evaluation, investigation

Customers are split before anything is fitted, so test customers never influence training. The supervised branch learns from theft labels; the unsupervised branch learns only what normal usage looks like. Both scores are evaluated on the same test customers and shown side by side in the investigation view.

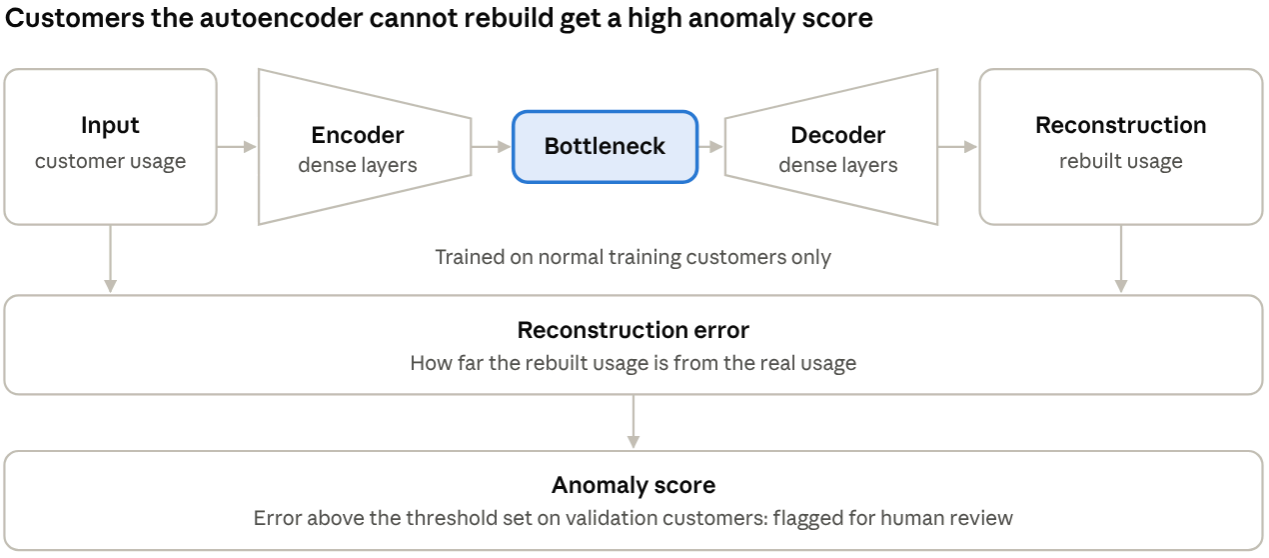
Low-level design

Training pipeline. Data is cleaned and customers are split before any model sees it. Classical baselines and deep models are trained independently on the same training customers, thresholds are chosen on validation customers, and the test set is used exactly once.



Training pipeline · 6 steps, baselines and deep models trained in parallel

Inside the autoencoder. The encoder squeezes a customer's usage into a small bottleneck, and the decoder tries to rebuild it. Because it only ever learns from normal customers, it rebuilds normal usage well and unusual usage badly. The size of that reconstruction error becomes the anomaly score. A high score means the usage is unusual, not that theft is proven.



Inside the autoencoder · compress, rebuild, measure the error

Workflow

The project is built as eight Jupyter notebooks, run in order, each saving what the next one needs.

1. **Data understanding and EDA.** Load the data, sort date columns chronologically, and study missing values, zeros, outliers, class imbalance and consumption trends.
2. **Preprocessing.** Fix the column order, treat the 3 Oct 2014 glitch as missing, cap extreme readings, impute gaps, and shuffle and split customers into train, validation and test sets (stratified, fixed seed). Imputers and scalers are fitted on training data only.
3. **Feature engineering.** Build per-customer features: mean, spread, zero-day ratio, longest zero run, sudden drops and spikes, volatility, and change from each customer's own baseline.
4. **Classical baselines.** Logistic Regression and Random Forest on the engineered features (supervised); Isolation Forest as an unsupervised anomaly detector.
5. **Deep learning models.** An LSTM classifier on the consumption sequence (aggregated weekly to keep training fast), and a dense autoencoder trained on normal training customers only, using reconstruction error as the anomaly score.
6. **Evaluation.** Every model scored on the same held-out test customers, with thresholds chosen on validation data.
7. **Explainability and risk analysis.** Feature importance and SHAP for tree models, reconstruction-error breakdowns for the autoencoder, and plots of each flagged customer's history.
8. **Interactive demo.** A notebook-based investigation view: pick a test customer and see their consumption, theft probability, anomaly score and the periods that looked unusual, using saved models.

Evaluation and impact

Every model is tested on the same held-out customers, and supervised and unsupervised models are compared within their own group, so each component's contribution is visible.

Model	Type	What it shows
Logistic Regression	Supervised, classical	Simplest baseline on engineered features
Random Forest	Supervised, classical	Strong non-deep baseline with feature importance

Model	Type	What it shows
LSTM	Supervised, deep learning	Whether learning directly from the sequence beats hand-made features
Isolation Forest	Unsupervised, classical	Anomaly detection without labels or deep learning
Dense autoencoder	Unsupervised, deep learning	Whether learned "normal" behaviour flags theft better than Isolation Forest

Metrics. Accuracy is misleading here: labelling everyone normal already scores 91.5%. We report precision, recall, F1, ROC-AUC, PR-AUC, false-positive rate and confusion matrices, plus precision among the top-ranked customers, which mirrors how a utility would choose whom to inspect. Labels are used for the unsupervised models only to evaluate them, never to train them.

Expected impact. A ranked, explained list of high-risk customers lets inspectors spend limited visits where theft is most likely, reducing revenue loss. Every flag is a lead for human investigation, not proof of theft.

Limitations. Labels are per customer with no theft date, and the dataset does not explain how they were assigned. Results come from one utility in one region and may not transfer elsewhere.

Future scope. Add a CNN on a weeks × days view of the data, try LSTM autoencoders and attention models, validate on a second public dataset, calibrate probabilities into a combined risk score, and add external factors such as weather and holidays.